

DAY 2: Instructor module, FRONTEND

Needs Day 1 done first: the instructor registration page uses `OtpService` (`services/otp.service.ts`) from the OTP day.

12 new files + 5 small edits. Three screens: the instructor registration and status page, the admin review page, and the instructor's course page.

1. The screens

Route	Who	What
<code>/instructor-register</code>	anyone (logged-out)	the full registration form: username, email, password, confirm, mobile, qualification, years of experience, expertise, bio, optional profile link. When OTP is on, after Submit the same page shows step 2 (enter the 6 digit code sent to the mobile) and step 3 (verified, Log in now); when OTP is off it goes to login as before
<code>/instructor-register</code>	a logged-in instructor	the status tracker : Submitted, Under review, Approved or Not approved, with the admin's reason and an "Update and resubmit" button when rejected; a link to "My courses" when approved
<code>/admin-instructors</code>	ADMIN	tabs Pending, Approved, Rejected, All with counts; Approve; Reject opens a box that needs a reason of 5 to 500 characters
<code>/instructor/courses</code>	INSTRUCTOR	the course list with Add, Edit, Delete (delete asks for confirmation). If not approved yet it explains why instead

The home page gets a **Become an instructor** button next to Register and Login. After registering, an instructor is sent to `/verify-phone` (Day 1) when OTP is on, or to login when it is off. When an instructor logs in they land on their status page.

2. Honest status

Built and tested in a real browser against a pretend backend following the same API: **31 checks, 0 failures** (home button, validation, pending page, admin reject needs a reason, instructor sees the reason, resubmit, approve, course add/edit/delete, role guards both ways, duplicate email message, OTP redirect, phone width, no JS errors). Not tested against the real Spring backend.

3. Edits to existing files

3a. `constant.ts`: inside `API`:

```
// instructors (Day 2)
INSTRUCTOR_REGISTER: '/api/instructor/register',
INSTRUCTOR_ME: '/api/instructor/me',
INSTRUCTOR_COURSES: '/api/instructor/courses',
ADMIN_INSTRUCTORS: '/api/admin/instructors',
```

3b. `app.module.ts`: import `InstructorModule` and place it **before** `AppRoutingModule`:

```
import { InstructorModule } from './modules/instructor/instructor.module';
// imports: [..., VerificationModule, InstructorModule, AppRoutingModule]
```

3c. components/login/login.component.ts: in `onSubmit`, directly after `next: (res: any) => { add:`

```
if (res.role === 'INSTRUCTOR') {
  this.loading = false;
  this.router.navigate(['/instructor-register']);
  return;
}
```

(If the Google guide's `goAfterLogin(res)` method exists, add the same five lines at the top of it too.)

3d. components/home/home.component.html: in the logged-out buttons (`<div class="cta" *ngIf="!role">`), add after the Login button:

```
<a mat-stroked-button class="big ghost" routerLink="/instructor-register">Become an instructor</a>
```

3e. components/navbar/navbar.component.html: in the desktop `<nav class="links">` add the new blocks, and the same links with `mat-menu-item` inside the `<mat-menu>`:

```
<!-- inside the ADMIN block, after Manage Admins -->
<a routerLink="/admin-instructors" routerLinkActive="active">Instructor Requests</a>

<!-- new block, after the ADMIN block -->
<ng-container *ngIf="role === 'INSTRUCTOR'">
  <a routerLink="/instructor-register" routerLinkActive="active">My application</a>
  <a routerLink="/instructor/courses" routerLinkActive="active">My courses</a>
</ng-container>
```

```
<!-- the same, inside <mat-menu> -->
<a mat-menu-item routerLink="/admin-instructors">Instructor Requests</a>
<ng-container *ngIf="role === 'INSTRUCTOR'">
  <a mat-menu-item routerLink="/instructor-register">My application</a>
  <a mat-menu-item routerLink="/instructor/courses">My courses</a>
</ng-container>
```

4. Check it (with the Day 2 backend)

1. Logged out: Home shows **Become an instructor**. Click it: the form opens.
2. Fill it with a 3 character bio and submit: it refuses. Fix and submit.
3. Log in as that instructor: the page says the application is under review.
4. Open "My courses": a note says you are not approved yet and there is no Add button.
5. Log in as admin, open **Instructor Requests**, press Reject: the button stays disabled until the reason has 5 characters. Reject with a reason.
6. Log back in as the instructor: the red box shows the reason; press **Update and resubmit**.
7. Admin approves. The instructor sees the green approved note and can add, edit and delete courses.
8. An instructor opening `/admin-instructors`, or an admin opening `/instructor/courses`, lands on the error page.

5. Troubleshooting

Symptom	Cause
<code>/instructor-register</code> shows the home page	<code>InstructorModule</code> after <code>AppRoutingModule</code>
Instructor lands on the customer dashboard after login	step 3c missed
Admin link missing	step 3e missed, or logged in as another role
403 "not approved" for an approved instructor	the browser still holds the old page data; reload

6. Complete code

`models/instructor.model.ts` (new file)

```

/** Shapes of the instructor API (see InstructorController and AdminInstructorController). */

export const INSTRUCTOR_ROLE = 'INSTRUCTOR';

export type ApplicationStatus = 'PENDING' | 'APPROVED' | 'REJECTED';

export interface InstructorDetails {
  qualification: string;
  experienceYears: number;
  expertise: string;
  bio: string;
  profileLink: string;
}

export interface InstructorRegisterRequest extends InstructorDetails {
  username: string;
  email: string;
  password: string;
  mobileNumber: string;
}

export interface InstructorView extends InstructorDetails {
  userId: number;
  username: string;
  email: string;
  mobileNumber: string;
  status: ApplicationStatus;
  rejectionReason: string | null;
  submittedAt: string | null;
  reviewedAt: string | null;
  reviewedBy: string | null;
}

```

`services/instructor.service.ts` (new file)

```

import { HttpClient, HttpParams } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';
import { Course } from '../models/course.model';
import { InstructorDetails, InstructorRegisterRequest, InstructorView } from '../models/instructor.model';

/** Instructor registration and application status, the admin's review, and an instructor's own courses. */
@Injectable({
  providedIn: 'root'
})
export class InstructorService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  // ----- the instructor

```

```

register(body: InstructorRegisterRequest): Observable<InstructorView> {
  return this.http.post<InstructorView>(`${this.apiUrl}${API.INSTRUCTOR_REGISTER}`, body);
}

me(): Observable<InstructorView> {
  return this.http.get<InstructorView>(`${this.apiUrl}${API.INSTRUCTOR_ME}`);
}

updateMe(details: InstructorDetails): Observable<InstructorView> {
  return this.http.put<InstructorView>(`${this.apiUrl}${API.INSTRUCTOR_ME}`, details);
}

myCourses(): Observable<Course[]> {
  return this.http.get<Course[]>(`${this.apiUrl}${API.INSTRUCTOR_COURSES}`);
}

addCourse(course: Course): Observable<Course> {
  return this.http.post<Course>(`${this.apiUrl}${API.INSTRUCTOR_COURSES}`, course);
}

updateCourse(courseId: number, course: Course): Observable<Course> {
  return this.http.put<Course>(`${this.apiUrl}${API.INSTRUCTOR_COURSES}/${courseId}`, course);
}

deleteCourse(courseId: number): Observable<void> {
  return this.http.delete<void>(`${this.apiUrl}${API.INSTRUCTOR_COURSES}/${courseId}`);
}

// ----- the admin
list(status: string = 'ALL'): Observable<InstructorView[]> {
  return this.http.get<InstructorView[]>(`${this.apiUrl}${API.ADMIN_INSTRUCTORS}`, { params: new
HttpParams().set('status', status) });
}

approve(userId: number): Observable<InstructorView> {
  return this.http.post<InstructorView>(`${this.apiUrl}${API.ADMIN_INSTRUCTORS}/${userId}/approve`, {});
}

reject(userId: number, reason: string): Observable<InstructorView> {
  return this.http.post<InstructorView>(`${this.apiUrl}${API.ADMIN_INSTRUCTORS}/${userId}/reject`, {
reason });
}
}

```

modules/instructor/instructor.module.ts (new file)

```

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SharedModule } from '../../shared/shared.module';
import { AuthGuard } from '../../components/authguard/authguard.guard';
import { ROLES } from '../../constant';
import { INSTRUCTOR_ROLE } from '../../models/instructor.model';
import { InstructorRegisterComponent } from '../../components/instructor-register/instructor-
register.component';
import { AdminInstructorsComponent } from '../../components/admin-instructors/admin-instructors.component';
import { InstructorCoursesComponent } from '../../components/instructor-courses/instructor-
courses.component';

const routes: Routes = [
  // public: the registration form, and (when an instructor is logged in) the application progress
  { path: 'instructor-register', component: InstructorRegisterComponent },
  { path: 'admin-instructors', component: AdminInstructorsComponent, canActivate: [AuthGuard], data: {
roles: [ROLES.ADMIN] } },
  { path: 'instructor/courses', component: InstructorCoursesComponent, canActivate: [AuthGuard], data: {
roles: [INSTRUCTOR_ROLE] } }
];

/**
 * Everything for instructors: registration and progress, the admin's approval page and the course CRUD.

```

```

* Import it in AppModule BEFORE AppRoutingModule so these routes are registered before the "" fallback.
*/
@NgModule({
  declarations: [InstructorRegisterComponent, AdminInstructorsComponent, InstructorCoursesComponent],
  imports: [SharedModule, RouterModule.forChild(routes)]
})
export class InstructorModule {}

```

components/instructor-register/instructor-register.component.ts (new file)

```

import { Component, OnDestroy, OnInit } from '@angular/core';
import { AbstractControl, FormBuilder, ValidationErrors, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';
import { InstructorService } from '../../services/instructor.service';
import { OtpService } from '../../services/otp.service';
import { INSTRUCTOR_ROLE, InstructorView } from '../../models/instructor.model';

type Mode = 'loading' | 'form' | 'status' | 'edit' | 'other-role';

/**
 * One page, two jobs. A visitor sees the instructor registration form. A logged-in instructor sees the
 * progress of
 * their application (Submitted, Under review, Decision), the admin's reason when it was rejected, and a
 * form to fix
 * and resubmit it. After approval the page points to "My courses".
 */
@Component({
  selector: 'app-instructor-register',
  templateUrl: './instructor-register.component.html',
  styleUrls: ['./instructor-register.component.css']
})
export class InstructorRegisterComponent implements OnInit, OnDestroy {
  mode: Mode = 'loading';
  profile?: InstructorView;
  loading = false;
  errorMessage = '';
  showPassword = false;
  otherRole = '';

  // ---- mobile OTP, shown inside this page (step 2) ----
  /** null = not known yet, true = the server asks for OTP, false = OTP is switched off on the server. */
  otpRequired: boolean | null = null;
  regStep: 'form' | 'verify' | 'done' = 'form';
  email = '';
  maskedMobile = '';
  code = '';
  sending = false;
  verifying = false;
  otpError = '';
  otpInfo = '';
  devCode = '';
  secondsLeft = 0;
  private timer?: any;

  private detailsControls = {
    qualification: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(100)]],
    experienceYears: [null as number | null, [Validators.required, Validators.min(0), Validators.max(60)]],
    expertise: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(150)]],
    bio: ['', [Validators.required, Validators.minLength(20), Validators.maxLength(1000)]],
    profileLink: ['', [Validators.pattern(/^$|^https?:\/\S+$/), Validators.maxLength(300)]]
  };

  registerForm = this.fb.group({
    username: ['', [Validators.required, Validators.maxLength(50), Validators.pattern(/^\S+$/)]],
    email: ['', [Validators.required, Validators.email, Validators.maxLength(120)]],
    password: ['', [Validators.required, Validators.pattern(/^(?=.*[A-Za-z])(?=.*\d){8,72}$/)]],
    confirmPassword: ['', Validators.required],
    mobileNumber: ['', [Validators.required, Validators.pattern(/^([0-9]{10})$/)]]
  });

```

```

    ...this.detailsControls
  }, { validators: (g: AbstractControl): ValidationErrors | null =>
    g.get('password')?.value === g.get('confirmPassword')?.value ? null : { mismatch: true } });

  editForm = this.fb.group({ ...this.detailsControls });

  constructor(private fb: FormBuilder, private instructors: InstructorService, private auth: AuthService,
    private router: Router, private otp: OtpService) {}

  ngOnInit(): void {
    if (!this.auth.isLoggedIn()) {
      this.mode = 'form';
      // only to show a short note and the step list; failing here changes nothing
      this.otp.status().subscribe({ next: s => (this.otpRequired = !!s.required), error: () =>
(this.otpRequired = null) });
      return;
    }
    const role = this.auth.getRole();
    if (role !== INSTRUCTOR_ROLE) { this.otherRole = String(role || '').toLowerCase(); this.mode = 'other-
role'; return; }
    this.loadStatus();
  }

  ngOnDestroy(): void { clearInterval(this.timer); }

  get f() { return this.registerForm.controls; }
  get e() { return this.editForm.controls; }

  /** 0 = submitted, 1 = under review, 2 = decision made (used to colour the tracker). */
  get step(): number {
    return this.profile?.status === 'PENDING' ? 1 : 2;
  }

  loadStatus(): void {
    this.mode = 'loading';
    this.instructors.me().subscribe({
      next: p => { this.profile = p; this.mode = 'status'; },
      error: () => { this.mode = 'status'; this.errorMessage = 'Could not load your application. Please
refresh the page.'; }
    });
  }

  submit(): void {
    if (this.registerForm.invalid || this.loading) { this.registerForm.markAllAsTouched(); return; }
    this.loading = true;
    this.errorMessage = '';
    const { confirmPassword, ...body } = this.registerForm.value as any;
    body.profileLink = (body.profileLink || '').trim();
    this.instructors.register(body).subscribe({
      next: () => this.afterRegister(body.email, body.mobileNumber),
      error: err => {
        this.loading = false;
        this.errorMessage = err.status === 409 ? 'A user with this email already exists'
          : err.status === 429 ? 'Too many attempts. Please wait a while and try again.'
          : err.status === 400 ? (err.error?.message || 'Please check the entered details')
          : 'Registration failed. Please try again later.';
      }
    });
  }

  startEdit(): void {
    if (!this.profile) { return; }
    const { qualification, experienceYears, expertise, bio, profileLink } = this.profile;
    this.editForm.reset({ qualification, experienceYears, expertise, bio, profileLink: profileLink || ''
});
    this.errorMessage = '';
    this.mode = 'edit';
  }

  cancelEdit(): void {
    this.mode = 'status';
  }

```

```

    this.errorMessage = '';
  }

  resubmit(): void {
    if (this.editForm.invalid || this.loading) { this.editForm.markAllAsTouched(); return; }
    this.loading = true;
    this.errorMessage = '';
    const v: any = this.editForm.value;
    this.instructors.updateMe({ ...v, profileLink: (v.profileLink || '').trim() }).subscribe({
      next: p => { this.loading = false; this.profile = p; this.mode = 'status'; },
      error: err => {
        this.loading = false;
        this.errorMessage = [400, 403].includes(err.status) ? (err.error?.message || 'Please check the
entered details')
          : 'Could not save your changes. Please try again.';
      }
    });
  }

  /** Application saved. If the server asks for OTP, stay on this page and show step 2; otherwise go to
login as before. */
  private afterRegister(email: string, mobile: string): void {
    this.otp.status().subscribe({
      next: s => {
        this.loading = false;
        this.otpRequired = !!s.required;
        if (!s.required) { this.router.navigate(['/login']); return; }
        this.email = email;
        this.maskedMobile = '*****' + String(mobile).slice(-4);
        this.regStep = 'verify';
        this.sendCode();
      },
      error: () => { this.loading = false; this.router.navigate(['/login']); }
    });
  }

  get canVerify(): boolean {
    return /^[0-9]{6}$/.test(this.code) && !this.verifying;
  }

  onCodeInput(): void {
    this.code = this.code.replace(/[^0-9]/g, '').slice(0, 6);
  }

  sendCode(): void {
    if (this.sending || this.secondsLeft > 0) { return; }
    this.sending = true;
    this.otpError = '';
    this.otp.send(this.email).subscribe({
      next: r => {
        this.sending = false;
        this.otpInfo = r.message;
        this.devCode = r.devCode || '';
        this.startCountdown(r.resendInSeconds || 30);
      },
      error: err => {
        this.sending = false;
        this.otpError = [400, 429].includes(err.status) ? (err.error?.message || 'Could not send the
code.')
          : 'The code could not be sent. Please try again in a moment.';
        if (err.status === 429) { this.startCountdown(Number(err.headers?.get('Retry-After')) || 30); }
      }
    });
  }

  verify(): void {
    if (!this.canVerify) { return; }
    this.verifying = true;
    this.otpError = '';
    this.otp.verify(this.email, this.code).subscribe({
      next: () => { this.verifying = false; clearInterval(this.timer); this.regStep = 'done'; },

```

```

    error: err => {
      this.verifying = false;
      this.code = '';
      this.otpError = [400, 429].includes(err.status) ? (err.error?.message || 'That code did not work.')
        : 'Could not check the code. Please try again.';
    }
  });
}

private startCountdown(seconds: number): void {
  clearInterval(this.timer);
  this.secondsLeft = seconds;
  this.timer = setInterval(() => {
    this.secondsLeft = Math.max(0, this.secondsLeft - 1);
    if (this.secondsLeft === 0) { clearInterval(this.timer); }
  }, 1000);
}
}

```

components/instructor-register/instructor-register.component.html (new file)

```

<div class="page narrow">

  <p class="muted center" *ngIf="mode === 'loading'">Loading...</p>

  <!-- someone else is logged in -->
  <section class="form-card center" *ngIf="mode === 'other-role'">
    <h2>Instructor registration</h2>
    <p class="lead">You are logged in as a {{ otherRole }}. Log out first if you want to register as an
instructor.</p>
    <a mat-stroked-button class="ghost" routerLink="/dashboard">Back to my dashboard</a>
  </section>

  <!-- the registration form -->
  <section class="wide" *ngIf="mode === 'form'">
    <ol class="steps" id="regSteps" aria-label="Registration steps" *ngIf="otpRequired">
      <li [class.on]="regStep === 'form'" [class.done]="regStep !== 'form'"><span>1</span>Your details</li>
      <li [class.on]="regStep === 'verify'" [class.done]="regStep === 'done'"><span>2</span>Verify
mobile</li>
      <li [class.on]="regStep === 'done'"><span>3</span>Done</li>
    </ol>

    <ng-container *ngIf="regStep === 'form'">
      <div class="page-head">
        <div>
          <h2>Become an instructor</h2>
          <p>Tell us about yourself. An administrator reviews every application. You can log in at any time
to follow its progress.</p>
        </div>
      </div>

      <p class="form-note" id="otpNote" *ngIf="otpRequired">After you submit we send a 6 digit code to your
mobile number. You verify it on the next step, here on this page.</p>
      <form [formGroup]="registerForm" (ngSubmit)="submit()" novalidate class="form-card grid">
        <h3 class="sect">Your account</h3>
        <mat-form-field appearance="outline">
          <mat-label>Username</mat-label>
          <input matInput id="username" formControlName="username" autocomplete="username" />
          <mat-error *ngIf="f['username'].errors?.['required']">Username is required</mat-error>
          <mat-error *ngIf="f['username'].errors?.['pattern']">Username cannot contain spaces</mat-error>
        </mat-form-field>
        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="email" />
          <mat-error *ngIf="f['email'].errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="f['email'].errors?.['email']">Enter a valid email</mat-error>
        </mat-form-field>
        <mat-form-field appearance="outline">
          <mat-label>Password</mat-label>

```

```

        <input matInput id="password" [type]="showPassword ? 'text' : 'password'"
formControlName="password" autocomplete="new-password" />
        <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword" [attr.aria-
label]="showPassword ? 'Hide password' : 'Show password'"><mat-icon>{{ showPassword ? 'visibility_off' :
'visibility' }}</mat-icon></button>
        <mat-hint>8 or more characters with a letter and a number</mat-hint>
        <mat-error *ngIf="f['password'].errors?.['pattern']">Use 8 to 72 characters with a letter and a
number</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline">
        <mat-label>Confirm password</mat-label>
        <input matInput id="confirmPassword" [type]="showPassword ? 'text' : 'password'"
formControlName="confirmPassword" autocomplete="new-password" />
        <mat-error *ngIf="registerForm.errors?.['mismatch']">Passwords do not match</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline">
        <mat-label>Mobile number</mat-label>
        <input matInput id="mobileNumber" inputmode="numeric" maxlength="10" formControlName="mobileNumber"
/>
        <mat-hint>We send a verification code to this number</mat-hint>
        <mat-error *ngIf="f['mobileNumber'].errors">Enter a 10 digit mobile number</mat-error>
    </mat-form-field>

    <h3 class="sect">About you</h3>
    <mat-form-field appearance="outline">
        <mat-label>Highest qualification</mat-label>
        <input matInput id="qualification" formControlName="qualification" />
        <mat-error *ngIf="f['qualification'].errors">Enter your qualification (2 to 100 characters)</mat-
error>
    </mat-form-field>
    <mat-form-field appearance="outline">
        <mat-label>Years of experience</mat-label>
        <input matInput id="experienceYears" type="number" min="0" max="60"
formControlName="experienceYears" />
        <mat-error *ngIf="f['experienceYears'].errors">Enter a number from 0 to 60</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
        <mat-label>Area of expertise</mat-label>
        <input matInput id="expertise" formControlName="expertise" />
        <mat-error *ngIf="f['expertise'].errors">Enter your area of expertise</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
        <mat-label>Short bio</mat-label>
        <textarea matInput id="bio" rows="4" maxlength="1000" formControlName="bio"></textarea>
        <mat-hint align="end">{{ f['bio'].value?.length || 0 }} / 1000 (at least 20)</mat-hint>
        <mat-error *ngIf="f['bio'].errors">Write at least 20 characters about your teaching
background</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
        <mat-label>LinkedIn or portfolio link (optional)</mat-label>
        <input matInput id="profileLink" formControlName="profileLink" placeholder="https://" />
        <mat-error *ngIf="f['profileLink'].errors">The link must start with http:// or https://</mat-error>
    </mat-form-field>

    <div class="form-error full" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
errorMessage }}</div>
    <div class="full actions">
        <button mat-flat-button color="primary" class="solid" id="submitApplication" type="submit"
[disabled]="loading">{{ loading ? 'Submitting...' : 'Submit application' }}</button>
        <span class="muted small">Already applied? <a routerLink="/login">Log in</a> to see the progress.
</span>
    </div>
</form>
</ng-container>

<!-- step 2: verify the mobile number (only when the server has OTP switched on) -->
<div class="form-card otp-card" *ngIf="regStep === 'verify'">
    <h2 id="stepVerify">Verify your mobile number</h2>
    <p class="lead">Your application is saved for <strong>{{ email }}</strong>. We sent a 6 digit code by
SMS to <strong>{{ maskedMobile }}</strong>. It works for 5 minutes.</p>
    <p class="form-note" *ngIf="devCode" id="devCodeNote">Development mode: no SMS was sent. Your code is

```

```

<strong id="devCode">{{ devCode }}</strong>.</p>
<div class="form-error" role="alert" *ngIf="otpError"><mat-icon>error_outline</mat-icon>{{ otpError
}}</div>
<form class="otp-form" (ngSubmit)="verify()" novalidate>
  <label class="field" for="otpCode">Verification code</label>
  <input id="otpCode" name="code" class="code" type="text" inputmode="numeric" autocomplete="one-
time-code"
    maxlength="6" placeholder="-----" [(ngModel)]="code" (ngModelChange)="onCodeInput()"
[disabled]="verifying" />
  <button mat-flat-button color="primary" class="solid" id="verifyOtp" type="submit"
[disabled]="!canVerify">{{ verifying ? 'Checking...' : 'Verify' }}</button>
</form>
<p class="muted small">Did not get it?
  <button type="button" class="link" id="resendOtp" (click)="sendCode()" [disabled]="sending ||
secondsLeft > 0">
    {{ secondsLeft > 0 ? 'Resend in ' + secondsLeft + 's' : (sending ? 'Sending...' : 'Resend code')
}}
  </button>
</p>
</div>

<!-- step 3 -->
<div class="form-card otp-card" *ngIf="regStep === 'done'">
  <div class="done-icon"><mat-icon>verified</mat-icon></div>
  <h2 id="regDone">Mobile number verified</h2>
  <p class="lead">Your application has been sent to an administrator. Log in any time to follow its
progress.</p>
  <a mat-flat-button color="primary" class="solid" id="goLogin" routerLink="/login">Log in now</a>
</div>
</section>

<!-- progress of the application -->
<section class="wide" *ngIf="mode === 'status' && profile">
  <div class="page-head">
    <div>
      <h2>Your instructor application</h2>
      <p>Hello {{ profile.username }}. Here is where your application stands.</p>
    </div>
  </div>

  <ol class="tracker" aria-label="Application progress">
    <li class="done"><span class="dot"><mat-icon>check</mat-icon></span><div><strong>Submitted</strong>
<span>{{ profile.submittedAt | date: 'd MMM y, h:mm a' }}</span></div></li>
    <li [class.done]="step >= 2" [class.now]="step === 1"><span class="dot"><mat-icon>{{ step >= 2 ?
'check' : 'hourglass_top' }}</mat-icon></span><div><strong>Under review</strong><span>{{ step >= 2 ?
'Reviewed' : 'An administrator is checking your details' }}</span></div></li>
    <li [class.ok]="profile.status === 'APPROVED'" [class.bad]="profile.status === 'REJECTED'"
[class.future]="profile.status === 'PENDING'">
      <span class="dot"><mat-icon>{{ profile.status === 'APPROVED' ? 'verified' : profile.status ===
'REJECTED' ? 'close' : 'flag' }}</mat-icon></span>
      <div><strong id="decision">{{ profile.status === 'APPROVED' ? 'Approved' : profile.status ===
'REJECTED' ? 'Not approved' : 'Decision' }}</strong>
        <span>{{ profile.reviewedAt ? (profile.reviewedAt | date: 'd MMM y, h:mm a') : 'You will see it
here' }}</span></div>
      </li>
    </ol>

    <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
errorMessage }}</div>

    <div class="note bad" id="rejectionNote" *ngIf="profile.status === 'REJECTED'">
      <strong>Reason from the administrator</strong>
      <p>{{ profile.rejectionReason }}</p>
      <button mat-flat-button color="primary" class="solid" id="editApplication" type="button"
(click)="startEdit()">Update and resubmit</button>
    </div>

    <div class="note ok" id="approvedNote" *ngIf="profile.status === 'APPROVED'">
      <strong>You are approved.</strong>
      <p>You can now add your courses and manage them.</p>
      <a mat-flat-button color="primary" class="solid" routerLink="/instructor/courses">Go to my

```

```

courses</a>
</div>

<div class="note" *ngIf="profile.status === 'PENDING'">
  <p>Reviews usually take a short while. You can fix a mistake in your details until a decision is
made.</p>
  <button mat-stroked-button class="ghost" id="editApplication" type="button"
(click)="startEdit()">Edit my details</button>
</div>

<dl class="summary form-card">
  <div><dt>Email</dt><dd>{{ profile.email }}</dd></div>
  <div><dt>Mobile</dt><dd>{{ profile.mobileNumber }}</dd></div>
  <div><dt>Qualification</dt><dd>{{ profile.qualification }}</dd></div>
  <div><dt>Experience</dt><dd>{{ profile.experienceYears }} years</dd></div>
  <div class="full"><dt>Expertise</dt><dd>{{ profile.expertise }}</dd></div>
  <div class="full"><dt>Bio</dt><dd>{{ profile.bio }}</dd></div>
  <div class="full" *ngIf="profile.profileLink"><dt>Link</dt><dd><a [href]="profile.profileLink"
target="_blank" rel="noopener noreferrer">{{ profile.profileLink }}</a></dd></div>
</dl>
</section>

<!-- fix and resubmit -->
<section class="wide" *ngIf="mode === 'edit'">
  <div class="page-head"><div><h2>Update your application</h2><p>Fix the details and send it for review
again.</p></div></div>
  <form [formGroup]="editForm" (ngSubmit)="resubmit()" novalidate class="form-card grid">
    <mat-form-field appearance="outline">
      <mat-label>Highest qualification</mat-label>
      <input matInput id="qualification" formControlName="qualification" />
      <mat-error *ngIf="e['qualification'].errors">Enter your qualification (2 to 100 characters)</mat-
error>
    </mat-form-field>
    <mat-form-field appearance="outline">
      <mat-label>Years of experience</mat-label>
      <input matInput id="experienceYears" type="number" min="0" max="60"
formControlName="experienceYears" />
      <mat-error *ngIf="e['experienceYears'].errors">Enter a number from 0 to 60</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
      <mat-label>Area of expertise</mat-label>
      <input matInput id="expertise" formControlName="expertise" />
      <mat-error *ngIf="e['expertise'].errors">Enter your area of expertise</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
      <mat-label>Short bio</mat-label>
      <textarea matInput id="bio" rows="4" maxLength="1000" formControlName="bio"></textarea>
      <mat-hint align="end">{{ e['bio'].value?.length || 0 }} / 1000 (at least 20)</mat-hint>
      <mat-error *ngIf="e['bio'].errors">Write at least 20 characters</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline" class="full">
      <mat-label>LinkedIn or portfolio link (optional)</mat-label>
      <input matInput id="profileLink" formControlName="profileLink" placeholder="https://" />
      <mat-error *ngIf="e['profileLink'].errors">The link must start with http:// or https://</mat-error>
    </mat-form-field>
    <div class="form-error full" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
errorMessage }}</div>
    <div class="full actions">
      <button mat-flat-button color="primary" class="solid" id="resubmitApplication" type="submit"
[disabled]="loading">{{ loading ? 'Saving...' : 'Save and send for review' }}</button>
      <button mat-stroked-button class="ghost" type="button" (click)="cancelEdit()">Cancel</button>
    </div>
  </form>
</section>
</div>

```

components/instructor-register/instructor-register.component.css (new file)

```

/* Instructor registration and progress (Midnight) */
.wide { display: block; }
.form-card.grid { max-width: none; display: grid; grid-template-columns: 1fr 1fr; gap: 6px 18px; }
.grid .full, .grid .sect, .grid .actions { grid-column: 1 / -1; }
.sect { margin: 10px 0 2px; font-size: 16px; color: var(--gold-text); }
.sect:first-child { margin-top: 0; }
.actions { display: flex; flex-wrap: wrap; align-items: center; gap: 14px; margin-top: 6px; }
.small { font-size: 13.5px; }

.tracker { list-style: none; margin: 0 0 22px; padding: 0; display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; }
.tracker li { display: flex; gap: 12px; align-items: flex-start; padding: 16px; border-radius: 14px; background: var(--surface); border: 1px solid var(--line); color: var(--muted); }
.tracker .dot { flex: none; display: grid; place-items: center; width: 34px; height: 34px; border-radius: 50%; border: 2px solid var(--line); }
.tracker .dot mat-icon { font-size: 19px; width: 19px; height: 19px; }
.tracker strong { display: block; color: var(--ink-2); font: 700 15px var(--display); }
.tracker span { font-size: 13px; }
.tracker li.done { border-color: rgba(52, 211, 153, .45); }
.tracker li.done .dot { background: var(--ok); border-color: var(--ok); color: #052e22; }
.tracker li.done strong { color: var(--ink); }
.tracker li.now { border-color: var(--gold); box-shadow: 0 0 3px rgba(245, 158, 11, .15); }
.tracker li.now .dot { border-color: var(--gold); color: var(--gold-text); }
.tracker li.now strong { color: var(--ink); }
.tracker li.ok { border-color: var(--ok); } .tracker li.ok .dot { background: var(--ok); border-color: var(--ok); color: #052e22; } .tracker li.ok strong { color: var(--ink); }
.tracker li.bad { border-color: var(--danger); } .tracker li.bad .dot { background: var(--danger); border-color: var(--danger); color: #3b0a0a; } .tracker li.bad strong { color: var(--ink); }

.note { margin-bottom: 20px; padding: 16px 18px; border-radius: 14px; background: var(--surface); border: 1px solid var(--line); }
.note p { margin: 6px 0 12px; color: var(--ink-2); line-height: 1.6; }
.note.bad { border-color: rgba(248, 113, 113, .5); background: rgba(248, 113, 113, .08); }
.note.bad strong { color: var(--danger); }
.note.ok { border-color: rgba(52, 211, 153, .5); background: rgba(52, 211, 153, .08); }
.note.ok strong { color: var(--ok); }

.summary { max-width: none; display: grid; grid-template-columns: 1fr 1fr; gap: 14px 24px; margin: 0; }
.summary div { min-width: 0; }
.summary .full { grid-column: 1 / -1; }
.summary dt { font: 700 12.5px var(--display); color: var(--gold-text); margin-bottom: 2px; }
.summary dd { margin: 0; color: var(--ink-2); overflow-wrap: anywhere; line-height: 1.55; }

/* step list and the inline OTP step */
.steps { display: flex; gap: 8px; margin: 0 0 18px; padding: 0; list-style: none; flex-wrap: wrap; }
.steps li { display: inline-flex; align-items: center; gap: 8px; padding: 6px 12px 6px 6px; border-radius: 999px; border: 1px solid var(--line); background: var(--surface-2); color: var(--muted); font: 600 13px Inter, sans-serif; }
.steps li span { display: grid; place-items: center; width: 22px; height: 22px; border-radius: 50%; background: var(--surface); border: 1px solid var(--line); font-size: 12px; }
.steps li.on { color: #1a1205; background: var(--gold); border-color: var(--gold); }
.steps li.on span { background: rgba(0, 0, 0, .15); border-color: transparent; color: #1a1205; }
.steps li.done { color: var(--ok); border-color: rgba(52, 211, 153, .45); }
.steps li.done span { background: rgba(52, 211, 153, .14); border-color: transparent; color: var(--ok); }
.otp-form { display: flex; flex-direction: column; gap: 12px; margin-top: 16px; }
.field { font: 700 13px var(--display); color: var(--gold-text); }
.code { width: 100%; padding: 14px; text-align: center; font: 700 28px/1 var(--display); letter-spacing: .5em; text-indent: .5em; color: var(--ink); background: var(--surface-2); border: 1px solid var(--line); border-radius: 12px; }
.code::placeholder { color: var(--muted); opacity: .5; }
.code:focus { outline: none; border-color: var(--gold); box-shadow: 0 0 0 3px rgba(245, 158, 11, .25); }
.link { border: 0; background: none; padding: 0; color: var(--gold-text); font-weight: 700; cursor: pointer; text-decoration: underline; text-underline-offset: 3px; }
.link.disabled { color: var(--muted); text-decoration: none; cursor: default; }
.done-icon { display: grid; place-items: center; width: 56px; height: 56px; margin: 0 0 14px; border-radius: 16px; background: rgba(52, 211, 153, .14); color: var(--ok); }
.done-icon mat-icon { font-size: 30px; width: 30px; height: 30px; }
.otp-card { max-width: 520px; }

@media (max-width: 720px) {

```

```
.form-card.grid, .summary { grid-template-columns: 1fr; }
.tracker { grid-template-columns: 1fr; }
}
```

components/admin-instructors/admin-instructors.component.ts (new file)

```
import { Component, OnInit } from '@angular/core';
import { InstructorService } from '../../services/instructor.service';
import { ApplicationStatus, InstructorView } from '../../models/instructor.model';

type Tab = 'PENDING' | 'APPROVED' | 'REJECTED' | 'ALL';

/**
 * Admin page: every instructor application with its details. The admin can approve it, or reject it with a
 * reason that the
 * instructor will read on their own application page.
 */
@Component({
  selector: 'app-admin-instructors',
  templateUrl: './admin-instructors.component.html',
  styleUrls: ['./admin-instructors.component.css']
})
export class AdminInstructorsComponent implements OnInit {
  readonly tabs: { key: Tab; label: string }[] = [
    { key: 'PENDING', label: 'Pending' }, { key: 'APPROVED', label: 'Approved' }, { key: 'REJECTED', label:
'Rejected' }, { key: 'ALL', label: 'All' }
  ];
  tab: Tab = 'PENDING';
  all: InstructorView[] = [];
  loaded = false;
  loadError = '';
  busyId: number | null = null;
  rejectingId: number | null = null;
  reason = '';
  actionError = '';
  actionOk = '';

  constructor(private instructors: InstructorService) {}

  ngOnInit(): void {
    this.load();
  }

  get shown(): InstructorView[] {
    return this.tab === 'ALL' ? this.all : this.all.filter(i => i.status === this.tab);
  }

  count(tab: Tab): number {
    return tab === 'ALL' ? this.all.length : this.all.filter(i => i.status === tab).length;
  }

  get reasonLength(): number {
    return this.reason.trim().length;
  }

  get reasonOk(): boolean {
    return this.reasonLength >= 5 && this.reasonLength <= 500;
  }

  load(): void {
    this.loadError = '';
    this.instructors.list('ALL').subscribe({
      next: list => { this.all = list; this.loaded = true; },
      error: () => { this.loaded = true; this.loadError = 'Could not load the applications. Please try
again.'; }
    });
  }

  approve(i: InstructorView): void {
```

```

    if (this.busyId !== null) { return; }
    this.busyId = i.userId;
    this.actionError = '';
    this.actionOk = '';
    this.instructors.approve(i.userId).subscribe({
      next: u => { this.replace(u); this.busyId = null; this.rejectingId = null; this.actionOk = u.username
+ ' is approved.'; },
      error: err => { this.busyId = null; this.actionError = err.error?.message || 'Could not approve.
Please try again.'; }
    });
  }

  startReject(i: InstructorView): void {
    this.rejectingId = i.userId;
    this.reason = '';
    this.actionError = '';
    this.actionOk = '';
  }

  cancelReject(): void {
    this.rejectingId = null;
    this.reason = '';
  }

  confirmReject(i: InstructorView): void {
    if (!this.reasonOk || this.busyId !== null) { return; }
    this.busyId = i.userId;
    this.actionError = '';
    this.instructors.reject(i.userId, this.reason.trim()).subscribe({
      next: u => { this.replace(u); this.busyId = null; this.rejectingId = null; this.reason = '';
this.actionOk = u.username + ' was rejected. The reason is saved.'; },
      error: err => { this.busyId = null; this.actionError = err.error?.message || 'Could not reject.
Please try again.'; }
    });
  }

  chip(status: ApplicationStatus): string {
    return status === 'APPROVED' ? 'ok' : status === 'REJECTED' ? 'bad' : 'wait';
  }

  trackById(_: number, i: InstructorView): number {
    return i.userId;
  }

  private replace(u: InstructorView): void {
    this.all = this.all.map(x => (x.userId === u.userId ? u : x));
  }
}

```

components/admin-instructors/admin-instructors.component.html (new file)

```

<div class="page">
  <div class="page-head">
    <div>
      <h2>Instructor requests</h2>
      <p>Review each application. A rejection needs a reason, and the instructor can read it and apply
again.</p>
    </div>
  </div>

  <div class="tabs" role="tablist">
    <button type="button" role="tab" *ngFor="let t of tabs" class="tab" [id]='''tab' + t.key"
[class.on]="tab === t.key" [attr.aria-selected]="tab === t.key" (click)="tab = t.key">
      {{ t.label }} <span class="n">{{ count(t.key) }}</span>
    </button>
  </div>

  <div class="form-error" role="alert" *ngIf="loadError"><mat-icon>error_outline</mat-icon>{{ loadError }}
</div>

```

```

<div class="form-error" role="alert" id="actionError" *ngIf="actionError"><mat-icon>error_outline</mat-
icon>{{ actionError }}</div>
<p class="form-note" id="actionOk" *ngIf="actionOk">{{ actionOk }}</p>

<p class="empty" *ngIf="loaded && !loadError && !shown.length">No {{ tab === 'ALL' ? '' : (tab |
lowercase) + ' ' }}applications.</p>

<div class="list">
  <article class="card" *ngFor="let i of shown; trackBy: trackById" [attr.id]='inst' + i.userId">
    <header>
      <div>
        <h3>{{ i.username }}</h3>
        <span class="muted">{{ i.email }} &middot; {{ i.mobileNumber }}</span>
      </div>
      <span class="chip" [ngClass]="chip(i.status)">{{ i.status | titlecase }}</span>
    </header>

    <dl>
      <div><dt>Qualification</dt><dd>{{ i.qualification }}</dd></div>
      <div><dt>Experience</dt><dd>{{ i.experienceYears }} years</dd></div>
      <div class="full"><dt>Expertise</dt><dd>{{ i.expertise }}</dd></div>
      <div class="full"><dt>Bio</dt><dd>{{ i.bio }}</dd></div>
      <div class="full" *ngIf="i.profileLink"><dt>Link</dt><dd><a [href]="i.profileLink" target="_blank"
rel="noopener noreferrer">{{ i.profileLink }}</a></dd></div>
      <div><dt>Applied</dt><dd>{{ i.submittedAt | date: 'd MMM y, h:mm a' }}</dd></div>
      <div *ngIf="i.reviewedAt"><dt>Reviewed</dt><dd>{{ i.reviewedAt | date: 'd MMM y, h:mm a' }}</dd>
    </div>
  </dl>

  <p class="reason" *ngIf="i.status === 'REJECTED' && i.rejectionReason"><strong>Reason given:</strong>
  {{ i.rejectionReason }}</p>

  <div class="actions" *ngIf="i.status !== 'APPROVED' && rejectingId !== i.userId">
    <button mat-flat-button color="primary" class="solid approve" type="button" (click)="approve(i)"
[disabled]="busyId !== null">
      {{ busyId === i.userId ? 'Working...' : 'Approve' }}
    </button>
    <button mat-stroked-button class="ghost danger reject" type="button" *ngIf="i.status === 'PENDING'"
(click)="startReject(i)" [disabled]="busyId !== null">Reject</button>
  </div>

  <div class="reject-box" *ngIf="rejectingId === i.userId">
    <label [for]="reason' + i.userId">Reason for rejection (the instructor will see this)</label>
    <textarea [id]="reason' + i.userId" class="reason-input" rows="3" maxlength="500"
[(ngModel)]="reason" placeholder="For example: please add your certificates and a link to your work">
    </textarea>
    <div class="count" [class.low]="!reasonOk">{{ reasonLength }} / 500 (at least 5)</div>
    <div class="actions">
      <button mat-flat-button class="solid danger-solid confirm-reject" type="button"
(click)="confirmReject(i)" [disabled]="!reasonOk || busyId !== null">
        {{ busyId === i.userId ? 'Saving...' : 'Reject application' }}
      </button>
      <button mat-stroked-button class="ghost" type="button" (click)="cancelReject()"
[disabled]="busyId !== null">Cancel</button>
    </div>
  </div>
</article>
</div>
</div>

```

components/admin-instructors/admin-instructors.component.css (new file)

```

/* Admin: instructor requests (Midnight) */
.tabs { display: flex; flex-wrap: wrap; gap: 8px; margin-bottom: 18px; }
.tab { display: inline-flex; align-items: center; gap: 8px; padding: 8px 16px; border-radius: 999px;
border: 1px solid var(--ink-2); background: var(--surface-2); color: var(--ink-2); font: 600 14px Inter,
sans-serif; cursor: pointer; }
.tab:hover { border-color: rgba(245, 158, 11, .5); }
.tab.on { background: var(--gold); border-color: var(--gold); color: #1a1205; }

```

```

.tab .n { padding: 1px 8px; border-radius: 999px; background: rgba(255, 255, 255, .12); font-size: 12.5px; }
.tab.on .n { background: rgba(0, 0, 0, .15); }
.empty { padding: 40px 0; text-align: center; color: var(--muted); }

.list { display: grid; grid-template-columns: repeat(auto-fill, minmax(420px, 1fr)); gap: 16px; }
.card { display: flex; flex-direction: column; gap: 12px; padding: 20px; border-radius: var(--radius); background: var(--surface); border: 1px solid var(--line); box-shadow: var(--shadow); }
.card header { display: flex; justify-content: space-between; gap: 12px; align-items: flex-start; }
.card h3 { font-size: 18px; margin-bottom: 2px; }
.muted { color: var(--muted); font-size: 13.5px; overflow-wrap: anywhere; }
.chip { flex: none; padding: 3px 12px; border-radius: 999px; font: 700 12.5px Inter, sans-serif; }
.chip.wait { background: var(--gold-soft); color: var(--gold-text); }
.chip.ok { background: rgba(52, 211, 153, .14); color: var(--ok); }
.chip.bad { background: rgba(248, 113, 113, .14); color: var(--danger); }
dl { display: grid; grid-template-columns: 1fr 1fr; gap: 10px 18px; margin: 0; }
dl div { min-width: 0; }
dl .full { grid-column: 1 / -1; }
dt { font: 700 12px var(--display); color: var(--gold-text); margin-bottom: 1px; }
dd { margin: 0; color: var(--ink-2); font-size: 14.5px; line-height: 1.5; overflow-wrap: anywhere; }
.reason { margin: 0; padding: 10px 12px; border-radius: 10px; background: rgba(248, 113, 113, .08); border: 1px solid rgba(248, 113, 113, .35); color: var(--ink-2); font-size: 14px; }
.reason strong { color: var(--danger); }
.actions { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 2px; }
.danger { color: var(--danger); border-color: rgba(248, 113, 113, .5); }
.danger-solid { background: var(--danger); color: #3b0a0a; }
.danger-solid[disabled] { opacity: .45; }
.reject-box { display: flex; flex-direction: column; gap: 6px; padding: 12px; border-radius: 12px; background: var(--surface-2); border: 1px solid var(--line); }
.reject-box label { font: 700 13px var(--display); color: var(--ink-2); }
.reason-input { width: 100%; resize: vertical; padding: 10px 12px; border-radius: 10px; border: 1px solid var(--line); background: var(--bg); color: var(--ink); font: 400 14.5px Inter, sans-serif; line-height: 1.5; }
.reason-input:focus { outline: none; border-color: var(--gold); }
.count { align-self: flex-end; font-size: 12.5px; color: var(--muted); }
.count.low { color: var(--danger); }
@media (max-width: 520px) { .list { grid-template-columns: 1fr; } dl { grid-template-columns: 1fr; } }

```

components/instructor-courses/instructor-courses.component.ts (new file)

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { MatDialog } from '@angular/material/dialog';
import { ConfirmDialogComponent } from '../confirm-dialog/confirm-dialog.component';
import { InstructorService } from '../services/instructor.service';
import { Course } from '../models/course.model';

/**
 * An approved instructor's own courses: list, add, edit and delete. The server decides who owns what, so
 * an instructor
 * can never change another person's course.
 */
@Component({
  selector: 'app-instructor-courses',
  templateUrl: './instructor-courses.component.html',
  styleUrls: ['./instructor-courses.component.css']
})
export class InstructorCoursesComponent implements OnInit {
  courses: Course[] = [];
  loaded = false;
  notApproved = false;
  loadError = '';
  formOpen = false;
  editingId: number | null = null;
  saving = false;
  formError = '';
  message = '';

  form = this.fb.group({

```

```

    courseType: ['', [Validators.required, Validators.maxLength(100)]],
    courseImageUrl: ['', [Validators.pattern(/^$|^https?:\/\/.+\/), Validators.maxLength(2000)]],
    courseDetails: ['', [Validators.required, Validators.maxLength(2000)]],
    coursePrice: [null as number | null, [Validators.required, Validators.min(0)]]
  });

  constructor(private fb: FormBuilder, private instructors: InstructorService, private dialog: MatDialog) {}

  ngOnInit(): void {
    this.load();
  }

  get f() { return this.form.controls; }

  load(): void {
    this.loadError = '';
    this.instructors.myCourses().subscribe({
      next: list => { this.courses = list; this.loaded = true; },
      error: err => {
        this.loaded = true;
        if (err.status === 403) { this.notApproved = true; } else { this.loadError = 'Could not load your courses. Please try again.'; }
      }
    });
  }

  openAdd(): void {
    this.editingId = null;
    this.form.reset({ courseType: '', courseImageUrl: '', courseDetails: '', coursePrice: null });
    this.formError = '';
    this.message = '';
    this.formOpen = true;
  }

  openEdit(c: Course): void {
    this.editingId = c.courseId ?? null;
    this.form.reset({ courseType: c.courseType, courseImageUrl: c.courseImageUrl || '', courseDetails: c.courseDetails, coursePrice: c.coursePrice ?? null });
    this.formError = '';
    this.message = '';
    this.formOpen = true;
    window.scrollTo({ top: 0 });
  }

  closeForm(): void {
    this.formOpen = false;
    this.editingId = null;
    this.formError = '';
  }

  save(): void {
    if (this.form.invalid || this.saving) { this.form.markAllAsTouched(); return; }
    this.saving = true;
    this.formError = '';
    const body = this.form.value as Course;
    const call = this.editingId === null ? this.instructors.addCourse(body) :
this.instructors.updateCourse(this.editingId, body);
    call.subscribe({
      next: () => {
        this.saving = false;
        this.message = this.editingId === null ? 'Course added.' : 'Course updated.';
        this.closeForm();
        this.load();
      },
      error: err => {
        this.saving = false;
        this.formError = [400, 403, 404].includes(err.status) ? (err.error?.message || 'Please check the entered details')
          : 'Could not save the course. Please try again.';
      }
    });
  }

```

```

    });
  }

  askDelete(c: Course): void {
    this.dialog.open(ConfirmDialogComponent, {
      data: { title: 'Delete course?', message: 'Delete "' + c.courseType + '"? This cannot be undone.',
confirmText: 'Yes, delete', cancelText: 'Cancel', icon: 'delete', danger: true }
    }).afterClosed().subscribe(yes => {
      if (!yes || c.courseId === undefined) { return; }
      this.instructors.deleteCourse(c.courseId).subscribe({
        next: () => { this.message = 'Course deleted.'; this.load(); },
        error: err => { this.message = ''; this.loadError = err.error?.message || 'The course could not be
deleted. Students may already be enrolled.'; }
      });
    });
  }

  trackById(_: number, c: Course): number | undefined {
    return c.courseId;
  }
}

```

components/instructor-courses/instructor-courses.component.html (new file)

```

<div class="page">
  <div class="page-head">
    <div>
      <h2>My courses</h2>
      <p>Add, edit and remove the courses you teach. Students see them in Available Courses.</p>
    </div>
    <button mat-flat-button color="primary" class="solid" id="addCourse" type="button" *ngIf="!notApproved"
(click)="openAdd()"><mat-icon>add</mat-icon>Add course</button>
  </div>

  <div class="form-note" id="notApproved" *ngIf="notApproved">
    Your instructor account is not approved yet, so you cannot manage courses.
    <a routerLink="/instructor-register">See your application</a>
  </div>

  <p class="form-note" id="courseMessage" *ngIf="message">{{ message }}</p>
  <div class="form-error" role="alert" *ngIf="loadError"><mat-icon>error_outline</mat-icon>{{ loadError }}
</div>

  <form class="form-card editor" *ngIf="formOpen" [formGroup]="form" (ngSubmit)="save()" novalidate>
    <h3>{{ editingId === null ? 'Add a course' : 'Edit course' }}</h3>
    <mat-form-field appearance="outline">
      <mat-label>Course title</mat-label>
      <input matInput id="courseType" formControlName="courseType" />
      <mat-error *ngIf="f['courseType'].errors">Enter the course title (up to 100 characters)</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline">
      <mat-label>Image link (optional)</mat-label>
      <input matInput id="courseImageUrl" formControlName="courseImageUrl" placeholder="https://" />
      <mat-error *ngIf="f['courseImageUrl'].errors">The link must start with http:// or https://</mat-
error>
    </mat-form-field>
    <mat-form-field appearance="outline">
      <mat-label>Details</mat-label>
      <textarea matInput id="courseDetails" rows="4" maxLength="2000" formControlName="courseDetails">
</textarea>
      <mat-error *ngIf="f['courseDetails'].errors">Enter the course details</mat-error>
    </mat-form-field>
    <mat-form-field appearance="outline">
      <mat-label>Price (INR)</mat-label>
      <input matInput id="coursePrice" type="number" min="0" formControlName="coursePrice" />
      <mat-error *ngIf="f['coursePrice'].errors">Enter a price of 0 or more</mat-error>
    </mat-form-field>
    <div class="form-error" role="alert" id="formError" *ngIf="formError"><mat-icon>error_outline</mat-
icon>{{ formError }}</div>
  </form>

```

```

    <div class="actions">
      <button mat-flat-button color="primary" class="solid" id="saveCourse" type="submit"
[disabled]="saving">{{ saving ? 'Saving...' : 'Save course' }}</button>
      <button mat-stroked-button class="ghost" type="button" (click)="closeForm()"
[disabled]="saving">Cancel</button>
    </div>
  </form>

  <p class="empty" id="noCourses" *ngIf="loaded && !notApproved && !loadError && !courses.length &&
!formOpen">You have not added a course yet. Choose "Add course" to start.</p>

  <div class="grid">
    <article class="card" *ngFor="let c of courses; trackBy: trackById" [attr.id]='course' + c.courseId">
      <div class="thumb"><img *ngIf="c.courseImageUrl" [src]="c.courseImageUrl" alt="" loading="lazy" />
<mat-icon *ngIf="!c.courseImageUrl">school</mat-icon></div>
      <div class="body">
        <h3>{{ c.courseType }}</h3>
        <p>{{ c.courseDetails }}</p>
        <div class="price">{{ c.coursePrice | currency: 'INR' : 'symbol' : '1.0-2' }}</div>
      </div>
      <div class="actions">
        <button mat-stroked-button class="ghost edit" type="button" (click)="openEdit(c)"><mat-
icon>edit</mat-icon>Edit</button>
        <button mat-stroked-button class="ghost danger del" type="button" (click)="askDelete(c)"><mat-
icon>delete</mat-icon>Delete</button>
      </div>
    </article>
  </div>
</div>

```

components/instructor-courses/instructor-courses.component.css (new file)

```

/* Instructor: my courses (Midnight) */
.editor { max-width: 640px; margin: 0 0 24px; }
.editor h3 { font-size: 20px; margin-bottom: 12px; }
.actions { display: flex; flex-wrap: wrap; gap: 10px; }
.empty { padding: 40px 0; text-align: center; color: var(--muted); }
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(290px, 1fr)); gap: 18px; }
.card { display: flex; flex-direction: column; gap: 12px; padding: 14px; border-radius: var(--radius);
background: var(--surface); border: 1px solid var(--line); box-shadow: var(--shadow); }
.thumb { height: 150px; border-radius: 10px; overflow: hidden; background: var(--surface-2); display: grid;
place-items: center; color: var(--muted); }
.thumb img { width: 100%; height: 100%; object-fit: cover; }
.thumb mat-icon { font-size: 44px; width: 44px; height: 44px; }
.body { flex: 1; }
.body h3 { font-size: 17px; margin-bottom: 6px; }
.body p { margin: 0 0 10px; color: var(--muted); font-size: 14px; line-height: 1.55; display: -webkit-box;
-webkit-line-clamp: 3; -webkit-box-orient: vertical; overflow: hidden; }
.price { font: 800 18px var(--display); color: var(--gold-text); }
.danger { color: var(--danger); border-color: rgba(248, 113, 113, .5); }

```